

Lab 5 — Capstone Code Hint Sheet

An End-to-End MS Mini-Project — three tracks, two blanks each

You do **one** track. Set `TRACK` to 'A', 'B', or 'C', then fill that track's **two** blanks below. Pick an option, type it in, and **run the cell** — the built-in `assert` checks tell you if your choice is structurally sound (shapes, split, frozen / trainable parameters, metrics in `[0, 1]`). They never check accuracy *magnitude*, so a small run still passes. Everything outside the marked blanks runs as-is.

Wrapping layers in a module (blanks A1 and B1). Those two blanks ask for a whole little `nn.Module` class — not just the layer list. Keep the skeleton exactly as below (same class **name** the blank asks for) and drop the option-(a) layers into the marked slot:

```
class ImprovedCNN(nn.Module):
    def __init__(self):
        class ROIClassifier(nn.Module):
            def __init__(self):
                super().__init__()
                self.features = <A1 conv layers>
                self.head = <A1 linear head>
                self.net = <B1 layers>
            def forward(self, x):
                return self.head(self.features(x))
        return self.net(x)
```

Track C's blank C1 is different — it is plain assignment statements that reuse the provided `TransferModel` wrapper, so option (a) there is already complete as written.

TRACK A — Beat the baseline

Blank A1 — ImprovedCNN. A *different* 1D-CNN than the Lab 2 baseline: deeper/wider conv blocks, an adaptive pool so the head size is fixed, one logit out.

- (a) `nn.Sequential(Conv1d(1,16,15,pad7),ReLU,MaxPool1d(4),Conv1d(16,32,15,pad7),ReLU,MaxPool1d(4),Conv1d(32,32,15,pad7),AdaptiveMaxPool1d(8),Flatten)` feeding `Linear(32*8,64) → Linear(64,1)`
 - (b) copy `BaselineCNN` unchanged (two blocks `1→8→16`, `Linear(16*375,32)→Linear(32,1)`)
 - (c) `nn.Linear(6000, 1)` only — no conv layers at all
- (a) is right: three wider blocks give a *different* trainable-parameter count (the check is `improved_params != baseline_params`) and still output `(N,1)`. (b) is an exact copy — same param count, so it **fails** the “must differ” check (and doesn’t “improve” anything). (c) throws away the peaks-are-local idea and, on a raw 6000-vector, is a weaker model, not a better CNN.

Blank A2 — augment. An m/z-jitter augmentation: nudge the spectrum a few bins along the mass axis and add a little noise. Must keep the shape and *change* the values.

- (a) `shift=randint(1,4); x=torch.roll(x,shift,dims=-1); return x+0.01*randn_like(x)`
 - (b) `return x` (no change)
 - (c) `return torch.zeros_like(x)`
- (a) shifts by 1–3 bins (always nonzero) and adds noise, so it passes both checks (`shape unchanged` and `not torch.allclose(aug, x)`). (b) returns the input untouched — fails the “must change the values” check and teaches the model nothing new. (c) destroys the spectrum entirely (all zeros) — technically “changed,” but it erases every peak, so training collapses.

TRACK B — Peak QC (real peak vs. noise)

Blank B1 — ROIClassifier. A small 1D-CNN reading one 256-point ROI `((N,1,256))` and returning one logit `(N,1)`.

- (a) `Conv1d(1,8,7,pad3),ReLU,MaxPool1d(4),Conv1d(8,16,7,pad3),ReLU,MaxPool1d(4),Flatten,Linear(16*16,32),ReLU,Linear(32,1)`
 - (b) `Conv1d(1,8,7,pad3),ReLU,MaxPool1d(4),Flatten,Linear(16*16,32),ReLU,Linear(32,2)`
 - (c) `Flatten,Linear(256,1)` directly (no conv)
- (a) is right: two pools take `256→64→16`, so `Linear(16*16,32)` matches and the output is `(N,1)` (the check is `out.shape == (2,1)`). (b) has *one* pool, so the flattened size is `8*64`, not `16*16` — a shape-mismatch crash — and `Linear(...,2)` outputs two columns, failing the `(N,1)` check (we use one logit + `BCEWithLogitsLoss`). (c) runs but is a bare linear model — allowed to try, yet it ignores the local peak shape the segment is about.

Blank B2 — pos.weight. Handle the imbalance (noise outnumbers peaks): up-weight the rare positive class by the `neg:pos` ratio, as a one-element tensor.

- (a) `pos_weight = torch.tensor([n_neg_train / n_pos_train])`
 (b) `pos_weight = torch.tensor([1.0])`
 (c) `pos_weight = torch.tensor([n_pos_train / n_neg_train])`
 (a) is right — noise:peak is about 2.3, so the weight is >1 (the check requires `pos_weight.item() > 1`) and `BCEWithLogitsLoss` then pays more attention to the rare peaks. (b) weights the classes equally — no imbalance handling — and fails the >1 check. (c) is the ratio *upside down* (<1), which *down*-weights the already-rare peaks — exactly wrong, and it fails the >1 check too.

TRACK C — New question, same spectra (transfer)

Blank C1 — freeze the body, attach a fresh head. Reuse the trained `saureus_model.features` as a *frozen* body, add a new head whose last layer outputs *one* logit, wrap in `TransferModel`.

- (a) `transfer_body = saureus_model.features; for p in transfer_body.parameters(): p.requires_grad = False; transfer_head = Sequential(Linear(saureus_model.feat_dim,32),ReLU,Linear(32,1)); transfer_model = TransferModel(transfer_body, transfer_head)`
 (b) same, but **omit** the `requires_grad = False` loop
 (c) same, but final layer is `Linear(32, 2)`
 (a) is right: freezing every body param makes `n_frozen == n_body` and leaves only the head trainable (`n_trainable == n_head`), and the `Linear(...,1)` head gives (N,1) logits — all three checks pass. (b) leaves the body trainable, so `n_trainable` includes the body and the `== n_head` check **fails** (and it isn't transfer learning any more — you'd retrain everything). (c) outputs two columns, failing the (N,1) check (this is a single-logit binary task).

Blank C2 — TRANSFER_LR. A *gentle* learning rate to fine-tune the fresh head on the frozen body — positive and $\leq 1e-2$.

- (a) `TRANSFER_LR = 5e-4`
 (b) `TRANSFER_LR = 0.5`
 (c) `TRANSFER_LR = 0`
 (a) is a good small step (smaller than the $1e-3$ the body was trained with); it passes $0 < \text{TRANSFER_LR} \leq 1e-2$. (b) is *huge* — it fails the $\leq 1e-2$ check and would blow the head up. (c) is zero — it fails the > 0 check and the head never learns anything.

*Every track ends the same way (Lecture 10): report **sensitivity, specificity, and AUROC + the confusion matrix** from **honest_eval** on the held-out split — never bare accuracy — and read the honest-error-analysis cell before you fill the rubric and your project one-pager.*